

RESEARCH PROJECT

COMPLETE ACTION PLAN

Comparative Study of Rule-Based vs Lightweight ML Lead Scoring in Low-Resource CRM Systems

Duration	8 Weeks
Team	2 Members
Stack	NestJS + FastAPI + React + PostgreSQL
Person A	Backend, ML, Research
Person B	Frontend, UI/UX, Research

Research Hypotheses — Keep Visible All 8 Weeks

H1

Accuracy: ML models (LR and RF) will achieve higher F1-score than rule-based scoring, but this advantage will shrink significantly at dataset sizes below 200 leads.

H2

Interpretability: Rule-based scoring provides perfect interpretability. LR offers partial interpretability via coefficients. RF offers the lowest interpretability despite highest accuracy.

H3

Operational Efficiency: Rule-based scoring will have the lowest latency and zero training cost. ML models will have measurably higher inference time and retraining overhead.

Every technical decision across 8 weeks should connect back to one of H1, H2, or H3. If you are doing something and cannot trace it to a hypothesis, stop and ask why.

8-Week Overview

Week	Focus	Person A (Backend)	Person B (Frontend)	Research	Key Deliverable
1	Setup	NestJS + Prisma + Docker	React + Tailwind + Redux	Lit review begins, intro draft	Running env + synthetic data
2	Core API	CRUD endpoints + Swagger	Lead table + filters	Problem statement + related work	Working API + Lead UI
3	Scoring (Rules)	ScoringService + events	Score badges + charts	Methodology 4.2 draft	Rule engine + scoring UI
4	ML + n8n	FastAPI LR+RF + n8n workflows	Comparison UI + notifications	Methodology 4.3–4.4	/compare endpoint + 3 workflows
5	Experiments	Sparsity experiment + analytics API	Analytics dashboard	Results tables + figures	Core research results generated
6	Integration	Full system integration	E2E flow + UI polish	Discussion + limitations	Integrated prototype + paper 70%
7	Testing	80%+ test coverage + load test	Cypress E2E + accessibility	Related work + paper draft	Full paper draft + test reports
8	Finalization	README + API docs + demo videos	UI polish + slides + demo	Abstract + IEEE format + submit	Final deliverables + paper ready

WEEK 1

Project Setup & Research Foundation

Environment, schema, synthetic data, literature review begins

Step-by-Step Actions

1

Read the full project plan end-to-end and map it to your role 🕒 1–1.5 hrs

WHY You cannot act on a plan you do not fully understand — this one hour prevents ten hours of rework.

DELIVERABLE A typed or handwritten list of: (a) everything YOU build in Weeks 1–4, (b) the three hypotheses written out, (c) what the final paper needs to prove.

OWNER Both

TIPS

- *Read once for overview, then again with a highlighter marking your tasks specifically.*
- *Write H1, H2, H3 on a sticky note and keep it visible — every decision should trace back to one of them.*

2

Install all core tools in the correct order 🕒 2–3 hrs

WHY A working local environment is the foundation everything depends on — without it, nothing runs.

DELIVERABLE Terminal confirms: `node --version`, `python --version`, `docker --version`, `git --version` all return version numbers without errors.

OWNER Both

TIPS

- *Install in this exact order: Git → Node.js v18+ → Python 3.10+ → Docker Desktop → VS Code.*
- *If Docker fails on Windows, enable WSL2 first (search 'Enable WSL2 Windows' on YouTube — 10 minutes).*

3

Read 5 paper abstracts + 2 full papers on lead scoring and rule-based vs ML classification 🕒 2–3 hrs

WHY Understanding existing literature tells you exactly what gap your paper fills — without this you cannot write a credible introduction.

DELIVERABLE A `research_notes.md` file: title + year + one-sentence summary for each of 5 abstracts, plus a 3–5 sentence summary for each of 2 full papers, plus a bullet noting what each paper does NOT cover.

OWNER Both

TIPS

- Search Google Scholar: 'lead scoring machine learning CRM', filtered 2019–2024. Read Abstract → Introduction → Conclusion first, then skim Methods.
- Register Zotero (zotero.org) and save every paper immediately — citation management from Day 1 saves hours later.

Starter Papers for Week 1 Literature Search

Search query 1: "lead scoring machine learning CRM"

- igi-global.com/chapter/ai-in-lead-scoring-and-crm/388281 — AI in Lead Scoring and CRM: Salesforce Einstein's Role
- frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1554325 — Frontiers in AI (2025)
- aisel.aisnet.org/hicss-53/da/machine_learning_in_finance/5 — HICSS ML in Finance
- ijcse.com/docs/INDJCSE22-13-02-098.pdf — IJCSE lead scoring with ML

Search query 2: "rule-based vs machine learning classification CRM"

- al-kindipublishers.org/index.php/jcsts/article/view/10444 — Rule-based vs ML classification
- ieeexplore.ieee.org/abstract/document/10425786 — IEEE rule-based vs ML comparison

Search query 3: "data sparsity small dataset classification"

- sciencedirect.com/science/article/abs/pii/S0957417412011529 — Data sparsity in classification
- link.springer.com/chapter/10.1007/11589990_16 — Small dataset ML challenges

4 Write a 200-word research gap statement ⌚ 1 hr

WHY Forcing yourself to articulate the gap reveals whether you actually understand the project — and becomes the seed of your Introduction section.

DELIVERABLE A research_question.md file: (1) research question, (2) what existing papers do, (3) what they do NOT do (the gap), (4) what your project contributes.

OWNER Both

TIPS

- Template: 'Most existing CRM lead scoring studies assume [large datasets / enterprise]. However, no study has examined [your gap]. This project addresses this by [your contribution].'
- Share with your teammate and ask: 'Does this match your understanding?' Misalignment caught in Week 1 is free — in Week 7 it costs days.

5

Initialise NestJS project + write the full Prisma schema with all research-critical fields ⌚ 3–4 hrs

WHY The database schema is the contract every other component depends on — getting it right in Week 1 avoids painful migrations throughout the project.

DELIVERABLE NestJS app starts (npm run start:dev no errors). schema.prisma contains all 6 tables. npx prisma migrate dev completes successfully. /test-db returns { leads: 0 }.

OWNER Person A

TIPS

- Run: `npx @nestjs/cli new crm-backend → npm install prisma @prisma/client → npx prisma init`. Paste schema, run `npx prisma migrate dev --name init`.
- If DB connection error: start Docker first with `docker-compose up -d postgres`, then retry. Research-critical fields: `ruleScore`, `mlScore`, `activeScore`, `scoreCategory`, `actuallyConverted` in leads table.

6

Write docker-compose.yml and get all three services running (PostgreSQL, n8n, FastAPI stub) ⌚ 2 hrs

WHY Containerising from Day 1 gives both teammates an identical environment — eliminating the 'works on my machine' problem.

DELIVERABLE `docker-compose up -d` runs without errors. Three URLs respond: localhost:5432 (PostgreSQL), localhost:5678 (n8n login page), localhost:8000/docs (FastAPI Swagger).

OWNER Person A

TIPS

- Copy `docker-compose.yml` from the project plan exactly. Run `docker-compose up -d` then `docker ps` to confirm all containers show 'Up'.
- FastAPI stub: create `/scoring-service/main.py` with just `app = FastAPI()` and one `@app.get('/health')` returning `{status: 'ok'}`. Real models come in Week 4.

7

Write the synthetic data generator (1000 leads + ground-truth conversion labels with sigma sensitivity analysis) ⌚ 3–4 hrs

WHY Your entire research experiment depends on this dataset — quality of synthetic data directly determines whether results are publishable.

DELIVERABLE `synthetic_leads.json` with 1000 leads, `actuallyConverted` boolean field, class balance 25–40% converted. THREE sigma variants generated: `sigma=5` (low noise), `sigma=10` (primary), `sigma=15` (high noise).

OWNER Person A

TIPS

- *pip install faker. Use weighted random.choices(): 40% Startup, 40% SME, 20% Enterprise. Implement determine_ground_truth() with Gaussian noise at sigma=10 as the primary experimental condition.*
- *CRITICAL: Generate all three sigma variants and save as synthetic_leads_s5.json, synthetic_leads_s10.json, synthetic_leads_s15.json. The sensitivity analysis (re-running sparsity experiment at all three sigma values) prevents reviewers from arguing results are an artefact of your noise choice rather than genuine model properties.*
- *Check balance: sum(1 for l in leads if l['actuallyConverted']) / len(leads). If below 15% or above 60%, adjust threshold — extreme imbalance makes ML results misleading.*

8

Initialise React app with Vite + Tailwind CSS + Redux Toolkit + folder structure ⌚ 2–3 hrs

WHY Setting up the frontend scaffold correctly in Week 1 means Person B is never blocked waiting for backend endpoints.

DELIVERABLE React app running at localhost:5173. Tailwind confirmed working (blue div test). Redux store configured with placeholder leads slice. Axios client pointing to http://localhost:3000/api.

OWNER Person B

TIPS

- *Run: npm create vite@latest crm-frontend -- --template react → npm install -D tailwindcss postcss autoprefixer → npx tailwindcss init -p. Follow Tailwind Vite guide at tailwindcss.com/docs/guides/vite.*
- *Create /mocks folder with JSON files for each API endpoint. Import directly in components this week — Lead Table can be built before the API is ready.*

9

Create wireframes for all four main dashboard screens ⌚ 2 hrs

WHY Wireframes force both teammates to agree on what the UI looks like before any code is written — preventing the most common source of wasted frontend time.

DELIVERABLE wireframes.pdf showing: (1) Lead List page, (2) Lead Detail modal with score breakdown, (3) Analytics Dashboard with all chart labels annotated with their API source, (4) Model Comparison view showing rule score vs ML score vs RF score side by side. Shared and approved by Person A.

OWNER Person B

TIPS

- *Use Figma (free) or pen + photo. The Model Comparison view MUST show all three scores side by side — this is the visual heart of your research contribution.*
- *Label every chart on the Analytics Dashboard with its data source (e.g. 'Score Distribution — from GET /api/analytics') so Person A knows exactly what endpoints to build in Week 5.*

10

End-of-week sync: demo environment + agree API contract ⌚ 1 hr

WHY Everything built in Weeks 2–8 depends on a shared API contract — one hour now prevents days of integration pain later.

DELIVERABLE `api_contract.md` committed to shared Git repo listing every agreed endpoint (method, path, request body, response shape). Both machines can run `docker-compose` up and hit the NestJS health endpoint.

OWNER Both

TIPS

- *Structure: 15 min both run `docker-compose` up; 20 min Person A walks through Prisma schema; 25 min together write Week 2 API contract.*
- *`api_contract.md` is the most important document this week. If Person A changes an endpoint shape, update the contract first — it is the source of truth.*

Week 1 Summary

#	Action	Time	Owner
1	Read the full project plan + map to your role	1–1.5 hr	Both
2	Install core tools: Git, Node.js, Python, Docker, VS Code	2–3 hr	Both
3	Read 5 paper abstracts + 2 full papers on lead scoring	2–3 hr	Both
4	Write 200-word research gap statement	1 hr	Both
5	NestJS project + full Prisma schema + migration	3–4 hr	Person A
6	Docker Compose: PostgreSQL + n8n + FastAPI stub	2 hr	Person A
7	Synthetic data generator (1000 leads + THREE sigma variants)	3–4 hr	Person A
8	React + Vite + Tailwind + Redux + folder structure	2–3 hr	Person B
9	Wireframes for all 4 dashboard screens	2 hr	Person B
10	End-of-week sync: demo env + API contract	1 hr	Both
	TOTAL	~19–24 hrs	

Week 1 End-of-Week Checklist

✓	Checkpoint	Owner
☐	<code>docker-compose</code> up runs without errors on both laptops	Both

☐	npx prisma migrate dev completes with all 6 tables created	Person A
☐	synthetic_leads.json exists with 1000 entries, class balance 25–40%	Person A
☐	THREE sigma variants generated: synthetic_leads_s5.json, _s10.json, _s15.json	Person A
☐	FastAPI stub returns 200 at localhost:8000/health	Person A
☐	localhost:5173 shows the React app with Tailwind styling confirmed	Person B
☐	wireframes.pdf shared and reviewed by both teammates	Person B
☐	research_notes.md has summaries for 2 full papers + 5 abstracts with gap notes	Both
☐	research_question.md states the gap in 200 words	Both
☐	api_contract.md is committed to the shared Git repository	Both
☐	Week 2 tasks assigned and both teammates know what to start Monday	Both

Step-by-Step Actions

1

Configure NestJS global pipeline: ValidationPipe + HttpExceptionHandler + LoggingInterceptor ⌚ 1–2 hrs

WHY This global wiring must happen before any endpoint is built — LoggingInterceptor is your primary H3 data collector.

DELIVERABLE main.ts registers all three globally. Every API response returns a consistent error shape. LoggingInterceptor console.logs method, path, statusCode, and latencyMs for every request.

OWNER [Person A](#)

TIPS

- Use useGlobalPipes, useGlobalFilters, useGlobalInterceptors in main.ts — not decorator-level. Test with an intentionally bad POST payload and confirm you get a structured 400 error.
- LoggingInterceptor uses RxJS tap() to record Date.now() before and after — this latency data feeds your H3 analysis in Week 5. Do not skip this.

2

Implement PrismaService + all ORM models as a NestJS injectable module ⌚ 2 hrs

WHY PrismaService is the database layer every other service will inject — it must be set up correctly before LeadsModule or ScoringModule can be built.

DELIVERABLE A PrismaModule that exports PrismaService. All Prisma models accessible. A smoke test endpoint confirms all 6 tables exist and are queryable.

OWNER [Person A](#)

TIPS

- Create prisma/prisma.service.ts extending PrismaClient, add OnModuleInit and OnModuleDestroy. Export it from PrismaModule with exports: [PrismaService].
- Run npx prisma studio to visually confirm all tables and columns are correct before writing any service logic.

3

Build LeadsModule: controller, service, and three DTOs ⌚ 3–4 hrs

WHY LeadsModule is the core of the entire CRM — it handles lead creation, retrieval, update, soft-delete, and interaction logging.

DELIVERABLE Five working endpoints. CreateLeadDto, UpdateLeadDto, FilterLeadDto all decorated with class-validator and @ApiProperty. POST /api/leads returns 201 with the created lead object. lead.created event fires on POST.

OWNER **Person A**

TIPS

- *Build in this order: DTOs first → Service (Prisma calls) → Controller (routes). Do not start the controller until the service is tested.*
- *Soft delete: never use `prisma.lead.delete()`. Set `status = LOST` or set a `deletedAt` field. Your `scoring_comparisons` table references lead IDs — hard deleting breaks referential integrity.*

4

Implement GET /api/leads with pagination + dynamic filter query building

🕒 2–3 hrs

WHY This is the most complex query of the week — it must support 4 filter dimensions and pagination while excluding soft-deleted records and logging latency.

DELIVERABLE GET /api/leads?status=HOT&page=1&limit=20 returns { data, total, page, limit }. Invalid filter values return 400. LoggingInterceptor records latency for every request (feeds H3).

OWNER **Person A**

TIPS

- *Build the Prisma where clause dynamically: only add a condition if the query param is defined. Use `Promise.all([findMany(), count()])` for parallel execution.*
- *Default `page=1`, `limit=20`, `max limit=100`. Enforce `@Max(100)` in `FilterLeadDto` — uncapped pagination can OOM your server during the sparsity experiment in Week 5.*

5

Configure Swagger via @nestjs/swagger and export OpenAPI spec as JSON

🕒 1–2 hrs

WHY Swagger documentation is both the working API spec for Person B and a deliverable in your paper's implementation section.

DELIVERABLE Swagger UI accessible at `localhost:3000/api/docs`. All endpoints visible with schemas. All DTOs decorated with `@ApiProperty`. JSON spec accessible at `/api/docs-json`.

OWNER **Person A**

TIPS

- *Install `@nestjs/swagger` and `swagger-ui-express`. Use `DocumentBuilder` to set `title`, `version`, `addBearerAuth()`. Set up `SwaggerModule.setup('api/docs', app, document)`.*
- *Decorate every controller method with `@ApiOperation()` and every DTO field with `@ApiProperty()`. This makes the spec immediately useful for Person B's Axios client.*

6

Build Lead List Table component with pagination controls and loading states

🕒 3–4 hrs

WHY The Lead List is the primary UI surface — building it in Week 2 with mock data means Person B is never blocked by API availability.

DELIVERABLE Lead List table renders all columns. Pagination controls work (prev/next, page number display). Loading skeleton shows during fetch. Empty state renders when no results. Row click opens stub drawer.

OWNER Person B

TIPS

- *Columns: Name, Email, Status badge (colour-coded), Source, Score, Score Category, Created date. Use /mocks/leads.json as the data source this week.*
- *Build loading skeletons as placeholder rows with animated shimmer. This prevents layout shift when real data loads later.*

7 Build Filter Sidebar with all four filter dimensions ⌚ 2–3 hrs

WHY The filter sidebar connects directly to the Redux filtersSlice — building it in Week 2 establishes the state management pattern used throughout the project.

DELIVERABLE Sidebar renders status checkboxes, source checkboxes, score range slider, and score category radio group. Apply and Reset buttons dispatch to Redux. Active filter count badge shows on sidebar toggle.

OWNER Person B

TIPS

- *Use Redux Toolkit's createSlice for filtersSlice. Filter changes update slice state immediately (pending) — only dispatch to the API when Apply is clicked. This prevents a request on every slider tick.*
- *Score range slider: use a dual-handle input or a simple min/max pair. The range 0–100 maps directly to the score field in the Prisma schema.*

8 Setup Redux store with three slices + RTK Query leadsApi ⌚ 2–3 hrs

WHY Redux Toolkit and RTK Query are the state and data-fetching layer — setting them up correctly in Week 2 makes Weeks 3–6 significantly faster.

DELIVERABLE Redux DevTools shows leadsSlice, filtersSlice, uiSlice. RTK Query leadsApi defined with endpoints for list and single lead. Filter changes in Redux automatically trigger RTK Query re-fetches.

OWNER Person B

TIPS

- *Create three slices: leadsSlice (list + selected + pagination meta), filtersSlice (active filter values), uiSlice (sidebar open, drawer open, loading flags).*
- *Wire RTK Query with createApi and configure the base URL from VITE_API_URL env var. This means switching from mock to real API requires only changing the env var.*

9

Add Axios interceptors for centralised auth injection and error handling ⌚

1–2 hrs

WHY Centralised error handling means every API error is caught and displayed consistently — preventing silent failures that are hard to debug later.

DELIVERABLE Request interceptor injects Bearer token. 401 responses clear auth state and redirect to login stub. 4xx errors dispatch a toast notification. 5xx errors show a generic server error toast.

OWNER **Person B**

TIPS

- Create an `axiosInstance` with `baseUrl` from env. Add request interceptor for auth header injection and response interceptor for error classification (401, 4xx, 5xx).
- Create a simple `useToast` hook used everywhere — this keeps error display consistent and saves time in Weeks 5–6 when debugging real API responses.

10

Research: Read 3 papers on rule-based classification + write Problem Statement draft + ADRs ⌚ 3–4 hrs

WHY Week 2 research tasks build directly toward Section 2 (Related Work) and Section 1 (Introduction) — doing them now prevents a writing bottleneck in Week 7.

DELIVERABLE `research_notes.md` updated with 3 new papers on rule-based classification. A 300-word Problem Statement draft saved as `paper/problem_statement.md`. ADR documents written for NestJS, Prisma, and soft delete choices.

OWNER **Both**

TIPS

- For Problem Statement: state the gap (SMBs lack affordable intelligent lead prioritisation), cite 2–3 statistics on sales rep time wasted on low-quality leads, frame the research question, outline the solution.
- ADR format: Context → Decision → Consequences. Write one ADR for NestJS choice, one for Prisma choice, one for soft delete decision. These go directly into Section 3 of the paper.

Week 2 API Endpoints Required

Method	Path	Description	Returns
POST	<code>/api/leads</code>	Create lead, fire <code>lead.created</code> event	201 + Lead object
GET	<code>/api/leads</code>	List with filters + pagination, exclude soft-deleted	<code>{ data, total, page, limit }</code>
GET	<code>/api/leads/:id</code>	Single lead with interaction history (eager loaded)	Lead + <code>interactions[]</code>

PUT	/api/leads/:id	Update lead fields (partial update via PartialType DTO)	200 + updated Lead
DELETE	/api/leads/:id	Soft delete — sets deletedAt timestamp	204 No Content
POST	/api/leads/:id/interactions	Log email, call, or meeting interaction	201 + Interaction
GET	/api/health	Health check for Docker monitoring — no auth required	{ status, uptime, timestamp }

Week 2 Summary

#	Action	Time	Owner
1	NestJS global pipeline: ValidationPipe + ExceptionFilter + LoggingInterceptor	1–2 hr	Person A
2	PrismaService + all ORM models injectable	2 hr	Person A
3	LeadsModule: controller, service, 3 DTOs + lead.created event	3–4 hr	Person A
4	GET /api/leads: pagination + dynamic filters	2–3 hr	Person A
5	Swagger setup + OpenAPI JSON export	1–2 hr	Person A
6	Lead List Table: columns + pagination + loading states	3–4 hr	Person B
7	Filter Sidebar: 4 filter dimensions + Redux integration	2–3 hr	Person B
8	Redux store: 3 slices + RTK Query leadsApi	2–3 hr	Person B
9	Axios interceptors: auth injection + error handling	1–2 hr	Person B
10	Research: 3 papers + Problem Statement draft + ADRs	3–4 hr	Both
	TOTAL	~20–27 hrs	

Week 2 End-of-Week Checklist

✓	Checkpoint	Owner
☐	All 7 API endpoints return correct status codes and response shapes	Person A
☐	Swagger UI accessible at localhost:3000/api/docs with all endpoints visible	Person A
☐	ValidationPipe returns 400 with field-level errors for invalid payloads	Person A

☐	LoggingInterceptor logs latencyMs for every request in the terminal (H3 data collection active)	Person A
☐	OpenAPI spec downloadable as JSON from /api/docs-json	Person A
☐	Lead List table renders with mock data and pagination controls work	Person B
☐	Filter sidebar dispatches to Redux and triggers table re-render on Apply	Person B
☐	Redux DevTools shows all three slices and state transitions	Person B
☐	Problem Statement draft (300 words) saved and shared in team folder	Both
☐	ADR documents written for NestJS, Prisma, and soft delete choices	Both
☐	api_contract.md updated with any endpoint shape changes from this week	Both

WEEK 3

Rule-Based Scoring Engine

DB-driven rules, event-driven scoring, unit tests

Step-by-Step Actions

1 Implement ScoringRulesService with DB-driven weight loading ⌚ 2–3 hrs

WHY Configurable DB-driven rule weights are a research contribution — they demonstrate that rule-based scoring can be adjusted without code deployment, which is a key advantage over ML retraining.

DELIVERABLE ScoringRulesService loads all active rules from the scoring_rules table. Weights can be changed in the DB and take effect on next score calculation without restarting the server. Documented as a contribution in paper/methodology.md.

OWNER Person A

TIPS

- Seed the scoring_rules table with the 10 default rules from the reference table below. These weights are your Section 4.2 scoring rules table.
- Add an in-memory cache with a 5-minute TTL so the DB is not queried on every scoring call. This also makes the latency measurement cleaner for H3.

2 Build ScoringService with EventEmitter listener for lead.created event ⌚ 3–4 hrs

WHY Event-driven architecture decouples lead creation from scoring — the CRM remains fast even if scoring takes time, and the pattern supports the n8n integration in Week 4.

DELIVERABLE ScoringService listens for lead.created events. When triggered, it calculates the rule score, updates the lead's ruleScore and scoreCategory fields, and saves a scoring_history entry with latencyMs.

OWNER Person A

TIPS

- Use NestJS EventEmitter2: @OnEvent('lead.created') in ScoringService. The event payload is the full Lead object. Scoring runs asynchronously so lead creation returns immediately.
- **CRITICAL:** Every calculateRuleScore() call must record start = Date.now() before and latencyMs = Date.now() - start after. Store in scoring_history. This is your H3 data — do not skip it.

3 Implement POST /api/score endpoint for manual re-scoring ⌚ 1–2 hrs

WHY Manual re-scoring lets you test the rule engine interactively and will be used in Week 5 during the sparsity experiment to batch-score all synthetic leads.

DELIVERABLE POST /api/score with a lead ID triggers a full re-score. Response includes the new ruleScore, scoreCategory, and the latencyMs. Documented in Swagger.

OWNER Person A

TIPS

- Accept both leadId (score an existing lead from DB) and a raw feature payload (score without saving — useful for the FastAPI comparison endpoint in Week 4).
- Test by POSTing a lead with clevel_title feature and confirming the score jumps by +30 as expected from the scoring_rules table.

4

Write unit tests with 80%+ coverage on the scoring module ⌚ 2–3 hrs

WHY Test coverage at 80%+ is a deliverable requirement AND it forces you to define and document the exact scoring logic, which you cite in the paper.

DELIVERABLE Jest runs with npm run test:cov showing 80%+ for scoring.service.ts, scoring-rules.service.ts, and calculateRuleScore. Edge cases tested: zero features, maximum score cap (100), missing rule weights.

OWNER Person A

TIPS

- Test structure: one describe block per scoring rule, testing that (a) the rule adds the correct points when the feature is present, and (b) the rule adds zero points when the feature is absent.
- Use Jest's jest.spyOn to mock PrismaService so tests run without a database connection. This makes tests fast and runnable in CI.

5

Build Score Badge component (HOT/WARM/COLD) and connect Lead Table to real API ⌚ 2–3 hrs

WHY Score badges are the primary visual signal of lead quality. Building them now with real API data validates the end-to-end flow.

DELIVERABLE Score Badge renders HOT (red), WARM (amber), COLD (blue) with correct colours. Lead Table fetches from GET /api/leads (not mock data). Badges update when score changes.

OWNER Person B

TIPS

- Score category thresholds: 0–39 = COLD (blue), 40–69 = WARM (amber), 70–100 = HOT (red). These should come from a constants file, not be hardcoded in the component.
- Switch from mock data to the real API by updating the RTK Query base URL env var. Keep the mock data file — you will need it for tests.

6

Create Score Distribution Pie/Donut Chart and Lead Detail Modal ⌚ 3–4 hrs

WHY The score distribution chart is Figure 3 in your paper. Building it in Week 3 means you can visually verify that your synthetic data has a realistic HOT/WARM/COLD distribution.

DELIVERABLE Score Distribution chart renders using Chart.js or Recharts. Lead Detail modal shows all lead fields + interaction history + score history timeline. Modal opens on table row click.

OWNER **Person B**

TIPS

- Use a donut chart with three segments: HOT (red), WARM (amber), COLD (blue). If your synthetic data shows 90% COLD, your scoring weights need tuning.
- Lead Detail modal: show interactions as a timeline (newest first). Show score history as a small line chart — this becomes the Score Drift visualisation in Week 5.

7

Research: Read 2 papers on feature engineering + write Methodology Section 4.2 draft ⌚ 2–3 hrs

WHY Section 4.2 documents your rule-based engine — writing it now while the code is fresh produces a better paper than writing it from memory in Week 7.

DELIVERABLE paper/methodology_4_2.md: full description of the rule-based scoring engine including the scoring rules table (all 10 rules with justifications), the configurable weights contribution, and the latency logging approach. Each rule cited to a paper or industry source.

OWNER **Both**

TIPS

- Structure of 4.2: (1) Overview of rule-based approach, (2) Feature categories (engagement vs demographic), (3) Scoring rules table with columns: Rule Name, Category, Points, Justification, (4) DB-driven weight system as contribution, (5) Latency measurement approach.
- Cite a paper for each major rule category. For example: 'form_fill receives +12 points reflecting its status as the strongest intent signal in B2B contexts (cite paper).' Search Google Scholar for supporting evidence.

Week 3 Scoring Rules Reference

Rule Name	Category	Points	Research Justification
base_score	Base	+10	All leads start with minimum intent signal — prevents zero-score edge cases
email_open	Engagement	+10 each	Direct behavioural signal of interest in B2B CRM literature
website_visit	Engagement	+5 each	Passive but trackable interest signal — lower weight than active actions

form_fill	Engagement	+12 each	Strongest intent signal — active action taken by prospect
clevel_title	Demographic	+30	Decision-maker = high conversion probability in B2B sales
vp_title	Demographic	+20	Senior stakeholder with budget authority
director_title	Demographic	+15	Influencer role in buying decisions
enterprise_size	Demographic	+20	Larger deal size potential
sme_size	Demographic	+10	Moderate deal potential
target_industry	Demographic	+15	TECH/FINANCE = primary target verticals in dataset

Week 3 Summary

#	Action	Time	Owner
1	ScoringRulesService with DB-driven weight loading + cache	2–3 hr	Person A
2	ScoringService with EventEmitter listener + latencyMs logging	3–4 hr	Person A
3	POST /api/score manual re-score endpoint	1–2 hr	Person A
4	Unit tests: 80%+ coverage on scoring module	2–3 hr	Person A
5	Score Badge component + Lead Table connected to real API	2–3 hr	Person B
6	Score Distribution chart + Lead Detail Modal	3–4 hr	Person B
7	Research: 2 papers on feature engineering + Methodology 4.2 draft	2–3 hr	Both
	TOTAL	~15–22 hrs	

Week 3 End-of-Week Checklist

✓	Checkpoint	Owner
☐	scoring_rules table seeded with all 10 default rules	Person A
☐	lead.created event triggers score calculation automatically on lead creation	Person A
☐	scoring_history table populates with latencyMs on every scoring event	Person A

☐	Unit tests pass with 80%+ coverage for the scoring module	Person A
☐	Score badges render with correct HOT/WARM/COLD colours on Lead List	Person B
☐	Lead Detail modal opens on row click showing interactions + score history	Person B
☐	Score Distribution chart renders with real data from the API	Person B
☐	Methodology Section 4.2 draft saved in team paper folder	Both
☐	Each scoring rule has a justification/citation documented for the paper	Both

WEEK 4

FastAPI Scoring Microservice + n8n Automation

LR + RF models, /compare endpoint, 3 n8n workflows

Step-by-Step Actions

1

Implement FastAPI /score/ml (Logistic Regression) and /score/rf (Random Forest) endpoints ⌚ 4–5 hrs

WHY These two endpoints are the core of your research comparison — LR provides a simpler, more interpretable ML baseline; RF provides maximum accuracy at the cost of interpretability.

DELIVERABLE POST /score/ml returns { score, confidence, featureImportance }. POST /score/rf returns the same shape. Both trained on startup using the 700-lead training split. Training time logged — this is your H3 data.

OWNER Person A

TIPS

- Feature vector: 9 features — emailOpens, websiteVisits, formFills (continuous), isClevel, isVP, isDirector (binary from jobTitle), isEnterprise, isSME (binary from companySize), isTechFinance (binary from industry). IDENTICAL for both models — controlled comparison.
- Use scikit-learn. On startup: load synthetic_leads_s10.json, split 70/30, fit LogisticRegression(max_iter=1000) and RandomForestClassifier(n_estimators=100). Log training time — this is your H3 data.

2

Implement POST /score/compare — the core research endpoint ⌚ 3–4 hrs

WHY /score/compare is the single most important endpoint in the entire project. It runs all three scoring methods and returns a side-by-side comparison including all latencies — this is what populates your scoring_comparisons research dataset.

DELIVERABLE POST /score/compare returns: { ruleScore, mlScore, rfScore, delta, agreement, ruleLatencyMs, mlLatencyMs, rfLatencyMs }. NestJS ScoringService calls this endpoint on every lead.created event and saves the result to scoring_comparisons.

OWNER Person A

TIPS

- Call all three methods internally with isolated timing: record Date.now() before each call, capture latencyMs after. Do not reuse the overall request time — each model needs its own measurement.
- agreement field: true if all three scores fall within the same category (HOT/WARM/COLD). delta: abs(ruleScore - max(mlScore, rfScore)). Both are research metrics, not just display fields.

3

Implement GET /research/metrics and POST /research/sparsity endpoints

⌚ 2–3 hrs

WHY `/research/metrics` is the endpoint you call in Week 5 to generate your paper's Table 2. `/research/sparsity` runs the full data sparsity experiment across 7 dataset sizes with 5 runs each.

DELIVERABLE GET `/research/metrics` returns P, R, F1, AUC-ROC per model on the 300-lead test set. POST `/research/sparsity` runs across [50, 100, 200, 300, 500, 700, 1000] leads with seeds [42, 7, 123, 256, 999] — returns mean $\pm$ std per size per model.

OWNER Person A

TIPS

- Load the 300-lead test split. For rule-based: compute score, threshold at 70 for HOT, evaluate against actuallyConverted ground truth. Same threshold for LR and RF.
- For `/research/sparsity`: each of the 7 sizes $\times$ 5 seeds = 35 independent runs. This takes a few minutes to run. Cache the result.

4 Build and export 3 n8n workflows as JSON ⌚ 3–4 hrs

WHY n8n workflows automate lead intake and follow-up — they justify the n8n component in the System Architecture section of the paper.

DELIVERABLE Three workflow JSON files in `/n8n-workflows/`: (1) `lead_intake.json` (Webhook $\rightarrow$ POST `/api/leads` $\rightarrow$ email), (2) `daily_followup.json` (Cron $\rightarrow$ stale leads $\rightarrow$ email), (3) `high_priority_alert.json` (HOT score $\rightarrow$ Slack/email). All exported and committed to the repo.

OWNER Person A

TIPS

- Access n8n at `localhost:5678`. Build workflows in the visual editor. Export each as JSON via the menu. The Webhook URL from `lead_intake.json` is used as the form submission target in Week 6.
- For the daily followup: use a Cron node (every day at 8 AM), an HTTP Request node calling `GET /api/leads?status=NEW`, and an Email node. This workflow is referenced in Section 3 of the paper.

5 Build Model Comparison toggle UI and Feature Importance mini-chart in Lead Detail ⌚ 3–4 hrs

WHY The Model Comparison view is the visual heart of your research contribution — it must show rule score vs ML score vs RF score side by side.

DELIVERABLE Lead Detail modal shows a toggle switching between Rule-Based, LR, and RF views. Each view shows the score, confidence (ML only), and a horizontal bar chart of feature importances. Score deltas highlighted in amber when disagreement $>$ 20 points.

OWNER Person B

TIPS

- *Feature importance bar chart: horizontal bars, one per feature, sorted by importance. For rule-based, importance = points awarded / total score. For LR/RF, use the featureImportance dict from the API response.*
- *Disagreement highlight: if $\text{abs}(\text{ruleScore} - \text{mlScore}) > 20$, show a warning badge: 'Models disagree significantly'. This is a UI demonstration of H1 — the accuracy gap is visible to the user.*

6

Research: Read 3 papers on LR vs RF + write Methodology 4.3 + 4.4 drafts

🕒 2–3 hrs

WHY Methodology sections 4.3 and 4.4 document your ML model choices — writing them while building the models produces accurate, detailed descriptions.

DELIVERABLE paper/methodology_4_3.md (LR: hyperparameters, feature vector, training approach) and paper/methodology_4_4.md (RF: hyperparameters, feature vector, comparison to LR). Table 1: Model hyperparameters draft complete.

OWNER Both

TIPS

- *For Table 1: columns are Model, Algorithm, Key Hyperparameters, Feature Vector Size, Training Dataset Size.*
- *Document why you chose the 70/30 train/test split: a fixed split gives more consistent latency measurements for H3 than cross-validation.*

FastAPI Endpoints Required This Week

Method	Path	Description	Returns
POST	/score/rules	Mirror NestJS rule engine (validation check — must match)	{ score, category, latencyMs }
POST	/score/ml	Logistic Regression scoring	{ score, confidence, featureImportance, latencyMs }
POST	/score/rf	Random Forest scoring	{ score, confidence, featureImportance, latencyMs }
POST	/score/compare	All three models + deltas + agreement	{ ruleScore, mlScore, rfScore, delta, agreement, all latencies }
GET	/research/metrics	Full evaluation on test split	{ precision, recall, F1, AUC-ROC } per model
POST	/research/sparsity	Sparsity experiment: 7 sizes × 5 seeds	{ results: [...] } mean ± std per size per model

Feature vector (9 features — identical for all models): emailOpens, websiteVisits, formFills (continuous); isClevel, isVP, isDirector (binary from jobTitle); isEnterprise, isSME (binary from companySize); isTechFinance (binary from industry).

Week 4 Summary

#	Action	Time	Owner
1	FastAPI: /score/ml (LR) and /score/rf (RF) endpoints	4–5 hr	Person A
2	FastAPI: POST /score/compare — all three models + latencies	3–4 hr	Person A
3	FastAPI: GET /research/metrics + POST /research/sparsity	2–3 hr	Person A
4	n8n: build + export 3 workflows as JSON	3–4 hr	Person A
5	Model Comparison UI + Feature Importance chart in Lead Detail	3–4 hr	Person B
6	Research: 3 papers + Methodology 4.3 + 4.4 drafts	2–3 hr	Both
	TOTAL	~17–23 hrs	

Week 4 End-of-Week Checklist

✓	Checkpoint	Owner
☐	/score/ml and /score/rf both return scores with featureImportance dict	Person A
☐	/score/compare returns all three scores + agreement + all latencies	Person A
☐	scoring_comparisons table populates on every lead creation event	Person A
☐	GET /research/metrics returns P, R, F1, AUC-ROC for all 3 models	Person A
☐	POST /research/sparsity runs 7 sizes × 5 seeds and returns mean ± std	Person A
☐	3 n8n workflow JSONs committed to /n8n-workflows/ in the repo	Person A
☐	Model Comparison toggle works in Lead Detail modal for all 3 models	Person B
☐	Feature importance bar chart renders for Rule, LR, and RF views	Person B
☐	Methodology 4.3 and 4.4 drafts saved in team paper folder	Both
☐	Table 1 (model hyperparameters) draft complete	Both

WEEK 5

Data Sparsity Experiment + Analytics

Core research experiment, analytics API, advanced UI — most important week

Step-by-Step Actions

1

Run the data sparsity experiment with 5 independent runs per size and sigma sensitivity analysis ⌚ 4–5 hrs

WHY This is the most important research result of the entire project — the sparsity curves show exactly at what dataset size ML begins to outperform rule-based scoring (your headline H1 finding).

DELIVERABLE Call `POST /research/sparsity.sparsity_experiment()` runs across sizes [50,100,200,300,500,700,1000] with 5 seeds [42,7,123,256,999]. Results saved as `/data/sparsity_results.json` with mean $\pm$ std per model per size. Run **ONCE** for each of the 3 sigma variants.

OWNER Person A

TIPS

- 5 independent runs are non-negotiable for publication credibility. Report mean $\pm$ standard deviation — reviewers will ask about variance. Error bars go on Figure 2.
- Run the experiment on ALL THREE sigma variants (s5, s10, s15) and compare the model rankings. If rankings are consistent across sigma values, this strengthens your findings. If they diverge, report the conditions where each model is preferred — this prevents reviewers arguing results are an artefact of your noise choice.

2

Build `GET /api/analytics` and `GET /api/analytics/research` endpoints ⌚ 3–4 hrs

WHY The analytics endpoints serve both the dashboard (for the demo) and the paper results section — building them now means Person B can build the charts immediately.

DELIVERABLE `GET /api/analytics` returns: total leads, score distribution breakdown, leads by source, leads by status, conversion rate. `GET /api/analytics/research` returns: scoring_comparisons summary, average latencies by model, agreement rate.

OWNER Person A

TIPS

- Add database indexes on score, status, createdAt, and industry before running analytics queries.
- `GET /api/analytics/research` is your paper's Table 3 data source. It should return a JSON structure you can paste directly into the results table without post-processing.

3

Run `GET /research/metrics` and record ALL numbers immediately ⌚ 1 hr

WHY This is a critical research milestone. Once you have real experimental results, you must record and version-control them immediately.

DELIVERABLE GET /research/metrics output saved to /data/metrics_sample.json. Values recorded in a shared spreadsheet or paper draft. Class balance, P, R, F1, AUC-ROC for all three models noted.

OWNER Both

TIPS

- Run the endpoint, copy the full JSON response, save to /data/metrics_sample.json, commit to Git immediately. Add a Git tag: `git tag -a v0.5-metrics 'First metrics run'`.
- If any result looks wrong (e.g. rule-based $F1 = 0.95$, which is implausibly high), do not proceed — debug the ground truth labelling in your synthetic data generator before continuing.

4 Generate Figure 2 (sparsity performance curves) with error bars 🕒 2–3 hrs

WHY The sparsity experiment results become Figure 2 in your paper — the performance curves showing all three models across dataset sizes, with error bars showing variance across seeds.

DELIVERABLE Figure 2 draft generated as matplotlib chart image saved to /data/figures/figure2_sparsity.png. Three lines (rule-based, LR, RF), x-axis = dataset size, y-axis = F1 score, error bars = std deviation.

OWNER Both

TIPS

- Generate the chart in Python using matplotlib with seaborn style. Plot three lines with error bars: `plt.errorbar(sizes, means, stds)`. Use distinct markers for each model.
- The crossover point (where ML lines cross the rule-based flat line) is your headline result. Annotate it with an arrow on the chart. If the lines never cross, report this finding — it may be equally publishable.

5 Build the Analytics Dashboard with 6+ charts 🕒 4–5 hrs

WHY The Analytics Dashboard is both a functional CRM feature and the visual demonstration of your research contribution — reviewers will form their first impression from this screen.

DELIVERABLE Analytics Dashboard page renders 6+ charts: score distribution, lead source breakdown, conversion funnel, model accuracy comparison, sparsity curve (Figure 2), and latency comparison. All connected to real API data.

OWNER Person B

TIPS

- Charts to build: (1) Score Distribution donut, (2) Lead Source breakdown bar, (3) Conversion Funnel, (4) Model Comparison radar/grouped bar, (5) Sparsity curve line chart with error bars (3 lines), (6) Latency comparison bar. Use Recharts or Chart.js — pick one and stick to it.

- The sparsity curve chart is the most important — three lines with error bars, dataset size on x-axis, F1 on y-axis. Annotate the crossover point. This is Figure 2 in your paper, visualised live.

6 Research: Start writing Results Section 5 with real numbers 🕒 3–4 hrs

WHY Writing results immediately after running experiments produces more accurate descriptions than writing from memory later. Week 5 is the only time you have fresh experimental context.

DELIVERABLE paper/results_5_1.md (H1 Accuracy results with Table 2 populated). paper/results_5_3.md (H3 Efficiency results with Table 3 populated). Table 2 and Table 3 complete with all experimental values.

OWNER Both

TIPS

- Table 2 structure: rows = Rule-Based / LR / RF, columns = Precision / Recall / F1 / AUC-ROC. Fill from /research/metrics output.
- Table 3 structure: rows = Rule-Based / LR / RF, columns = Avg Training Time / Avg Inference (ms) / Std Dev / Projected Cost (10k leads/month). Fill from latency measurements. Do NOT include expected ranges.

Data Sparsity Experiment Design

Dataset Size	Train (70%)	Test (30%)	Independent Runs	Report
50 leads	35	15	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
100 leads	70	30	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
200 leads	140	60	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
300 leads	210	90	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
500 leads	350	150	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
700 leads	490	210	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model
1000 leads	700	300	5 (seeds: 42, 7, 123, 256, 999)	mean F1 ± std per model

Run the sparsity experiment on ALL THREE sigma variants (s5, s10, s15). If model rankings are consistent across all three, your findings are robust. If they diverge, report the conditions under

which each model is preferred. This sensitivity analysis goes in an Appendix and prevents reviewers from rejecting your results.

Week 5 Summary

#	Action	Time	Owner
1	Data sparsity experiment: 7 sizes × 5 runs × 3 sigma variants	4–5 hr	Person A
2	GET /api/analytics + GET /api/analytics/research endpoints	3–4 hr	Person A
3	Run /research/metrics and record results to Git immediately	1 hr	Both
4	Generate Figure 2 sparsity curves with error bars	2–3 hr	Both
5	Analytics Dashboard: 6+ charts connected to real API	4–5 hr	Person B
6	Results Section 5: write 5.1 + 5.3 with real numbers	3–4 hr	Both
	TOTAL	~17–22 hrs	

Week 5 End-of-Week Checklist

✓	Checkpoint	Owner
☐	/data/sparsity_results.json committed with mean ± std for all 7 sizes × 3 models	Person A
☐	Sigma sensitivity analysis run: sparsity results for sigma=5, 10, 15 all saved	Person A
☐	/data/metrics_sample.json committed with P, R, F1, AUC-ROC for all 3 models	Both
☐	Figure 2 sparsity curve with error bars saved to /data/figures/figure2_sparsity.png	Both
☐	Analytics Dashboard renders 6+ charts connected to live API data	Person B
☐	Sparsity curve line chart with error bars visible on dashboard	Person B
☐	Table 2 (accuracy comparison) complete with real experimental values	Both
☐	Table 3 (latency comparison) complete with real measured values — no pre-specified ranges	Both
☐	Results Section 5.1 and 5.3 drafts saved in paper folder	Both

☐	Database indexes added on score, status, createdAt, industry	Person A
--------------------------	--	----------

WEEK 6

End-to-End Integration + Results Finalization

Full system working, canonical research data locked in

Step-by-Step Actions

1

Connect all n8n workflows to the production NestJS API and test end-to-end ⌚ 3–4 hrs

WHY Integration is where the system either works or reveals the bugs that were hidden when each service was tested in isolation.

DELIVERABLE Full flow tested: n8n Webhook → POST /api/leads → lead.created event → /score/compare → scoring_comparisons saved → HOT score triggers n8n alert. All three n8n workflows firing correctly.

OWNER Person A

TIPS

- Test the Lead Intake workflow first: trigger the webhook manually with a test payload and confirm a lead appears in the database with scores populated.
- Document every integration bug found and fixed in a *BUGS.md* file. This becomes the Bug Fix Documentation deliverable and shows reviewers your QA discipline.

2

Lock in canonical research results — call metrics once, save, version-control, never re-run ⌚ 1–2 hrs

WHY Canonical results are your experimental record. Locking them in this week prevents data snooping — re-running experiments after seeing unfavourable results is a research integrity issue.

DELIVERABLE GET /research/metrics output saved to /data/results_canonical.json on the full 300-lead test set. POST /research/sparsity output saved to /data/sparsity_canonical.json. Both files committed with tag v0.6-canonical. DO NOT re-run after this.

OWNER Both

TIPS

- Run each endpoint *ONCE*. Copy the full JSON response. Save to /data/. Run: `git add data/ && git commit -m 'lock canonical results v0.6' && git tag v0.6-canonical`
- If results look surprising, discuss and document the interpretation — do not re-run to get 'better' results. Surprising findings (e.g. rule-based beats ML) are often *MORE* publishable than expected findings.

3

Write Discussion Section — interpret H1, H2 (formal interpretability checklist), H3 ⌚ 3–4 hrs

WHY The Discussion section is where you explain what your results mean — it is the section reviewers read most carefully and the one most students write last and worst.

DELIVERABLE paper/discussion.md: separate paragraphs for H1 interpretation, H2 interpretability checklist results (Table 5 complete), H3 efficiency interpretation, and a Limitations paragraph.

OWNER Both

TIPS

-
-

H2 Interpretability Checklist — Table 5 in Section 5.2

Score each model across exactly 3 dimensions, using the scoring scale 1 = fully satisfied, 0.5 = partially satisfied, 0 = not satisfied. This operationalises interpretability as a structured, reproducible assessment rather than a subjective judgment.

Dimension	Rule-based	Logistic Regression	Random Forest
Global Explainability — can the model's overall decision logic be fully described without running predictions?	1.0	0.5	0.0
Local Explainability — can the score for any individual lead be fully attributed to input features?	1.0	0.5	0.0
Feature Attribution Availability — does the model natively output per-feature contribution scores without post-hoc approximation?	1.0	1.0	1.0
TOTAL SCORE	3.0 / 3.0	2.0 / 3.0	1.0 / 3.0

H3 CRITICAL: For Table 3, report ONLY the measured average latency per model. Do NOT include expected or estimated ranges. Pre-specifying ranges (e.g. 'rule-based ~1–5ms') risks confirmation bias — you may unconsciously collect data that confirms the range. Measure inference latency for each model across all test leads, compute mean and standard deviation, and present these as empirical findings. Let the numbers speak without anchoring readers or yourself to prior expectations.

4

Draft all paper figures (Figure 1: Architecture, Figure 2: Sparsity, Figure 3: Confusion Matrices) ⌚ 3–4 hrs

WHY Figures in an IEEE paper carry the same weight as tables — they must be clear, labelled, and reproducible from the code in the repository.

DELIVERABLE Figure 1: System architecture diagram. Figure 2: Sparsity performance curves (finalized with canonical data + error bars). Figure 3: Confusion matrices for all three models side by side. All saved as PNG to /data/figures/.

OWNER Both

TIPS

- Figure 1: draw in draw.io (free at draw.io) and export as PNG. Show the five layers: React → NestJS (via REST) → PostgreSQL + FastAPI → n8n. Label each arrow with the protocol/event name.
- Figure 3 confusion matrices: use scikit-learn's ConfusionMatrixDisplay on the canonical test set predictions. Create a 1×3 subplot (one per model). Label axes: Predicted / Actual. Add class labels: Converted / Not Converted.

5 Full user flow testing + UI polish + cross-browser testing ⌚ 3–4 hrs

WHY The prototype must work end-to-end for the Week 7 demo and the Week 8 demo video — any bugs found in Week 6 are much cheaper to fix than in Week 8.

DELIVERABLE Complete user flow works: create lead → see it in list → open detail → view score → see comparison → trigger n8n alert. UI tested in Chrome, Firefox, and Safari. Toast notifications and loading states polished.

OWNER Person B

TIPS

- Test the complete research demonstration flow: (1) create a lead with C-level title, (2) observe it auto-scored as HOT, (3) open Lead Detail and view the Rule vs ML vs RF comparison, (4) verify the n8n alert fired. This is your demo script.
- Add keyboard shortcuts: 'N' for new lead form, 'I' for search bar, 'F' to toggle filter sidebar. Document them in a help tooltip.

Week 6 Summary

#	Action	Time	Owner
1	Connect all n8n workflows + test full end-to-end flow	3–4 hr	Person A
2	Lock in canonical results: metrics + sparsity, commit + tag	1–2 hr	Both
3	Discussion Section: H1, H2 interpretability checklist, H3 (measured latency only)	3–4 hr	Both
4	Draft Figures 1, 2 (with error bars), 3 and save to /data/figures/	3–4 hr	Both
5	Full user flow testing + UI polish + cross-browser	3–4 hr	Person B
	TOTAL	~13–18 hrs	

Week 6 End-of-Week Checklist

✓	Checkpoint	Owner
☐	/data/results_canonical.json committed and tagged v0.6-canonical	Both
☐	/data/sparsity_canonical.json committed and tagged v0.6-canonical	Both
☐	Full end-to-end flow: lead creation → scoring → n8n alert all working	Person A
☐	BUGS.md documents all integration issues found and fixed	Person A
☐	Discussion Section draft saved with H1, H2, H3 interpretations	Both
☐	Table 5 (interpretability checklist) complete with 3-dimension scores for all 3 models	Both
☐	H3 latency results: only measured values in Table 3, no pre-specified ranges	Both
☐	Figure 1 (architecture), Figure 2 (sparsity with error bars), Figure 3 (confusion matrices) all drafted	Both
☐	UI works in Chrome, Firefox, and Safari without layout breaks	Person B
☐	Paper is approximately 70% complete (Sections 1–5 drafted)	Both

Step-by-Step Actions

1 Integration tests for all API endpoints using Jest + Supertest ⌚ 4–5 hrs

WHY Integration tests catch bugs that unit tests miss — they test the actual HTTP layer and database interactions. 80%+ coverage is a deliverable and a marker of engineering rigour in the paper.

DELIVERABLE Jest integration test suite passes. Coverage report shows 80%+ for all API endpoints. Edge cases covered: invalid IDs, missing required fields, filter combinations, pagination boundaries.

OWNER [Person A](#)

TIPS

- *Structure: one test file per controller (leads.controller.spec.ts, score.controller.spec.ts). Use a test database or in-memory SQLite via prisma-mock.*
- *Critical test cases: POST /api/leads with missing required field → 400. GET /api/leads/:id with non-existent ID → 404. DELETE /api/leads/:id → confirm deletedAt is set, not the row deleted. POST /score/compare → confirm scoring_comparisons row created.*

2 Load testing + security hardening ⌚ 2–3 hrs

WHY Load testing validates H3. Security hardening is a required deliverable — untreated vulnerabilities undermine the paper's claim that the system is production-viable.

DELIVERABLE Load test report with avg response time, p95, p99 at 100 concurrent users saved as /data/load_test_report.json. Security issues resolved: input sanitisation, security headers (helmet), rate limiting.

OWNER [Person A](#)

TIPS

- *Use k6 (k6.io — free): write a script simulating 100 virtual users each making 10 requests. Target GET /api/leads and POST /score/compare.*
- *Security checklist: install @nestjs/throttler for rate limiting, install helmet for HTTP headers, review all string inputs for injection risk. Also optimise Docker container size: use node:18-alpine base image and multi-stage builds.*

3 pytest for FastAPI: test all /score/* and /research/* endpoints ⌚ 2–3 hrs

WHY FastAPI endpoints are the core of your ML contribution — untested ML endpoints are a research credibility risk, as a bug could silently produce wrong metrics.

DELIVERABLE pytest suite passes for all 6 FastAPI endpoints. Tests verify: correct score range (0–100), featureImportance keys match the 9 feature vector, /compare returns all three scores, /research/metrics returns valid F1 and AUC-ROC values.

OWNER [Person A](#)

TIPS

- Use FastAPI's `TestClient`. Create a test fixture that loads the models once and reuses them across all test functions — loading models takes ~2s each run.
- Test for determinism: call `/score/ml` with the same input twice and confirm you get the same output. Stochastic results indicate a bug in model loading.

4

Cypress E2E tests + accessibility audit + mobile responsive fixes + performance profiling ⌚ 3–4 hrs

WHY E2E tests are your quality safety net for the demo. Accessibility, mobile, and performance are required deliverables that reviewers expect to see in a final prototype.

DELIVERABLE Cypress test suite with 15+ passing scenarios. WCAG 2.1 Level A accessibility audit completed. Mobile responsive fixes applied. Bundle size optimised with code splitting. Loading skeletons + performance profiling complete.

OWNER [Person B](#)

TIPS

- 15 flows to cover: create lead, lead appears in list, status filter works, pagination next/prev, lead detail opens on click, rule vs ML toggle, score badge colour, analytics charts render, sparsity chart shows 3 lines, empty state, toast on API error, filter count badge updates, lead score updates after re-score, n8n alert received for HOT lead, responsive layout at 768px.
- For bundle size: add `React.lazy()` and `Suspense` for the Analytics page. For performance: use Chrome DevTools Lighthouse audit and fix any LCP issues. For accessibility: install `axe-core` and run in test mode.

5

Write Related Work section (Section 2) — cite 15+ papers ⌚ 4–5 hrs

WHY Related Work is the section most likely to be challenged by reviewers — it must demonstrate that you know the field, have read recent work, and have correctly identified the gap.

DELIVERABLE `paper/related_work.md`: 600–800 words covering 5 topic clusters with 15+ citations. Each cluster ends with a sentence explaining why existing work does not fully address your research question.

OWNER [Both](#)

TIPS

- Five clusters: (1) Lead Scoring ML — 4–5 papers, (2) Rule-Based vs ML Classification — 3–4 papers, (3) Microservices Architecture — 2–3 papers, (4) Data Sparsity / Small Datasets — 2–3 papers, (5) CRM Automation — 2 papers. Search Google Scholar with date filter: 2019–2024. Aim for at least 3 papers from 2022+.

- *For each paper: one sentence on what they did + one sentence on their limitation. E.g.: 'Freitas et al. (2021) demonstrated ML superiority in enterprise CRM lead scoring but assumed dataset sizes exceeding 10,000 leads, making their findings inapplicable to SME contexts.'*

6

Write System Architecture section (Section 3) and assemble full paper draft

🕒 3–4 hrs

WHY Section 3 is where you justify your technology choices — your opportunity to demonstrate that the architecture was designed to answer the research question, not just to build a cool system.

DELIVERABLE paper/system_architecture.md complete. Full paper draft assembled as a single Overleaf project (all sections 1–7 present, even if rough). Sent to supervisor or peer reviewer for feedback before Week 8.

OWNER Both

TIPS

- *Section 3 structure: (1) Architecture overview with Figure 1, (2) Core API (NestJS + Prisma + PostgreSQL) — justify each choice with an ADR reference, (3) Scoring Microservice (FastAPI + scikit-learn) — justify separation from NestJS, (4) Automation Layer (n8n), (5) Data Flow narrative.*
- *Use the ADR documents you wrote in Week 2 as the source for technology choice justifications — do not rewrite them, reference and summarise them in the narrative.*

Week 7 Related Work: Papers to Cite by Topic

Topic Cluster	Target Count	Example Authors / Keywords
Lead Scoring ML	4–5 papers	Freitas et al., Fluss et al., recent CRM ML papers 2019–2024
Rule-Based vs ML Classification	3–4 papers	Interpretability vs accuracy trade-off literature
Microservices Architecture	2–3 papers	Justifying NestJS + FastAPI split, microservices CRM
Data Sparsity / Small Datasets	2–3 papers	Few-shot learning, small data ML challenges, class imbalance
CRM Automation + Workflow	2 papers	n8n or similar orchestration, workflow automation in CRM

Week 7 Summary

#	Action	Time	Owner
1	Integration tests: Jest + Supertest, 80%+ coverage	4–5 hr	Person A

2	Load testing: k6 at 100 concurrent users + security hardening + Docker optimisation	2–3 hr	Person A
3	pytest: all FastAPI /score/* and /research/* endpoints	2–3 hr	Person A
4	Cypress E2E (15+ scenarios) + accessibility audit + mobile fixes + performance profiling	3–4 hr	Person B
5	Related Work Section 2: 15+ citations, 5 clusters	4–5 hr	Both
6	System Architecture Section 3 + full paper draft assembled	3–4 hr	Both
	TOTAL	~18–24 hrs	

Week 7 End-of-Week Checklist

✓	Checkpoint	Owner
☐	Backend test coverage report shows 80%+ for all modules	Person A
☐	Load test report saved with p95 latency for key endpoints	Person A
☐	Security headers (helmet) + rate limiting configured	Person A
☐	Docker container size optimised with alpine base image	Person A
☐	pytest passes for all 6 FastAPI endpoints	Person A
☐	Cypress: 15+ E2E scenarios all passing in headless mode	Person B
☐	WCAG 2.1 Level A accessibility audit completed — issues fixed	Person B
☐	Mobile responsive fixes applied and tested at 768px	Person B
☐	Bundle size optimised with code splitting (React.lazy)	Person B
☐	Performance profiling complete (Lighthouse audit score noted)	Person B
☐	Related Work section drafted with 15+ citations in 5 clusters	Both
☐	System Architecture section complete with Figure 1 reference	Both
☐	Full paper draft assembled (all sections 1–7 present, even if rough)	Both
☐	Paper draft sent to supervisor or peer for feedback	Both

Step-by-Step Actions

1

Write comprehensive README + API integration guide + n8n workflow documentation + Docker deployment guide ⌚ 3–4 hrs

WHY The README is the first thing anyone reads when they visit your GitHub repository — it must enable a complete stranger to run your project in under 15 minutes.

DELIVERABLE README.md covers: project overview, quick start (5 steps), full setup guide, environment variables reference, Docker deployment, API endpoint reference, n8n workflow setup. API integration guide with examples for every endpoint. n8n workflow docs with screenshots + JSON explained.

OWNER Person A

TIPS

- *Structure: (1) Project Overview + research context, (2) Quick Start (5 commands), (3) Architecture diagram embed, (4) Environment Variables table, (5) Full Setup Guide, (6) API Endpoints table, (7) n8n Workflows documentation with screenshots.*
- *Test the README by following it on a clean machine or in a fresh Docker container. If any step fails, fix it. The goal is zero-friction reproducibility.*

2

Record demo videos: backend (5 min) + frontend (5 min) + combined project showcase (10 min) ⌚ 2–3 hrs

WHY The demo videos are the primary way your project will be evaluated by people who do not run the code. THREE videos are required per the edited plan.

DELIVERABLE Three video recordings: (1) Backend demo video (5 min) — API endpoints, Swagger, scoring pipeline, n8n workflows, (2) Frontend demo video (5 min) — lead list, filters, model comparison, analytics dashboard, (3) Project showcase video (10 min, narrated end-to-end) — full research demonstration flow.

OWNER Both

TIPS

- *Use OBS Studio (free) or Loom for screen recording. Write a script for each video first — every minute should be planned. Rehearse once before recording.*
- *Key moment to emphasise: the model comparison toggle showing the rule score vs ML vs RF score side by side. This is the visual proof of your research contribution. Pause on it and explain what the viewer is seeing.*

3

Write User Manual (Person B) and Handover Notes (Person B) ⌚ 2–3 hrs

WHY The user manual enables non-technical stakeholders to use the prototype. Handover notes enable future developers to continue the project — both are deliverables in the edited plan.

DELIVERABLE User manual with annotated screenshots covering: creating a lead, reading the score, using filters, interpreting the model comparison, and reading the analytics dashboard. Handover notes documenting architecture decisions, known issues, and suggested next steps.

OWNER **Person B**

TIPS

- *User manual: 8–12 annotated screenshots in a PDF or Notion page. Each screenshot annotated with numbered callouts. Keep language non-technical — write for a sales manager, not a developer.*
- *Handover notes: document the three areas most likely to confuse a new developer: (1) Prisma schema research fields and their purpose, (2) the scoring comparison data flow, (3) how to re-run the sparsity experiment with new data.*

4

Write the Abstract (150–200 words) and Conclusion + Future Work (Section 7) ⌚ 2–3 hrs

WHY The Abstract is written last because it summarises results you now have. The Conclusion ties every section together — both are short but high-impact sections.

DELIVERABLE paper/abstract.md: 150–200 words following the template below.
paper/conclusion.md: 300–400 words covering summary of findings, practical implications, and 3–5 future work directions.

OWNER **Both**

TIPS

- *Abstract is written LAST — only after all experimental results are final. Use the template below: fill in your actual H1 and H3 findings from the canonical results.*
- *Future Work: (1) Real-world lead data replacing synthetic, (2) Additional ML models (XGBoost, neural network), (3) Online learning for real-time model updates, (4) Multi-tenant CRM evaluation, (5) A/B testing framework for scoring approaches in production.*

Abstract Template — Fill in After Week 6 Canonical Results

Lead scoring in small and medium enterprise CRM systems is traditionally performed through manually configured rule-based approaches. While machine learning alternatives offer potential accuracy gains, their practical advantage over rule-based systems in low-resource, data-sparse environments remains underexplored. This paper presents a comparative study of three lead scoring approaches — rule-based, logistic regression, and random forest — implemented within a microservices CRM prototype using NestJS and FastAPI. We conduct data sparsity experiments across dataset sizes from 50 to 1,000 synthetic leads with ground-truth conversion labels, measuring accuracy (F1, AUC-ROC), interpretability, and operational cost for each approach. Our results show [FILL IN YOUR ACTUAL H1 AND H3 FINDINGS HERE]. We propose a cost-benefit

framework for practitioners choosing scoring approaches in low-resource CRM deployments and release our prototype and evaluation code as open-source.

5

Format paper to IEEE template in Overleaf + final proofreading pass ⌚ 4–5 hrs

WHY IEEE format is specific and non-negotiable for submission — margins, column layout, font sizes, and citation style must all match the template exactly.

DELIVERABLE Paper formatted in Overleaf using the IEEE Conference template. All sections in the correct order. All figures numbered and captioned correctly. All citations in IEEE format. Word count: 6–8 pages (without references). Final project report (15–20 pages) also completed.

OWNER Both

TIPS

- Overleaf has an IEEE Conference template — search 'IEEE Conference Proceedings' in the template gallery. Use `\cite{}` for all citations — build the `.bib` file from Zotero export.
- Proofreading checklist: every figure referenced in the text, every table has a caption, all abbreviations defined on first use (LR, RF, CRM, SME, F1, AUC-ROC), no section ends with a dangling sentence. The final project report (15–20 pages) includes all implementation details and can be longer/less formal than the IEEE paper.

6

Create 15-slide presentation deck + identify target conference + clean the repository ⌚ 3–4 hrs

WHY The presentation deck is evaluated in person. Conference selection determines where your work gets seen. A clean repository is a professional signal that the work is complete and reproducible.

DELIVERABLE 15-slide deck complete and reviewed. Conference identified with deadline confirmed. GitHub repository has clean commit history, no debug logs, inline comments added, `.env.example` committed. Submission package assembled.

OWNER Person B+Both

TIPS

- Slide structure: Slides 1–3 (motivation + gap), Slides 4–5 (architecture), Slides 6–7 (scoring approaches), Slides 8–9 (experiment design), Slides 10–11 (results — most important), Slides 12–13 (discussion + limitations), Slides 14–15 (conclusion + future work).
- Repository cleanup: remove all `console.log` debug statements, add JSDoc comments to all public functions, confirm `.env` is gitignored and `.env.example` is committed. Run `git log` to review commit messages.

Week 8 Summary

#	Action	Time	Owner
---	--------	------	-------

1	README + API guide + n8n workflow docs + Docker deployment guide	3–4 hr	Person A
2	Three demo videos: backend (5min) + frontend (5min) + showcase (10min)	2–3 hr	Both
3	User manual with annotated screenshots + Handover notes	2–3 hr	Person B
4	Abstract (150-200 words, written last) + Conclusion + Future Work	2–3 hr	Both
5	IEEE format in Overleaf + final proofreading + Final project report (15-20 pages)	4–5 hr	Both
6	15-slide presentation + conference identified + repo cleanup + submission package	3–4 hr	Both
	TOTAL	~16–22 hrs	

Week 8 Final Submission Checklist

✓	Checkpoint	Owner
☐	README enables a stranger to run the project in under 15 minutes	Person A
☐	All debug console.log statements removed from codebase	Person A
☐	.env.example committed (never .env with real credentials)	Person A
☐	n8n workflow documentation: screenshots + JSON files explained	Person A
☐	Backend demo video (5 min) recorded and link shared	Person A
☐	Frontend demo video (5 min) recorded and link shared	Person B
☐	Project showcase video (10 min, narrated end-to-end) recorded and link shared	Both
☐	User manual (annotated screenshots) complete — written for non-technical users	Person B
☐	Handover notes (architecture + known issues + next steps) committed to repo	Person B
☐	Abstract (150–200 words) written using template — fills in actual experimental findings	Both
☐	Paper formatted to IEEE template in Overleaf — 6–8 pages without references	Both
☐	All figures numbered and captioned, all tables have titles	Both
☐	All citations in IEEE format, .bib file complete in Zotero	Both

☐	15-slide presentation deck complete and reviewed by both teammates	Both
☐	Target conference identified with submission deadline confirmed	Both
☐	GitHub repository has clean commit history and inline comments	Both
☐	Final project report (15–20 pages) complete	Both
☐	Submission package prepared (PDF + cover letter if required)	Both

Target Submission Venues

Venue	Type	Scope	Fit	Difficulty
IEEE ICMLA	Conference	Machine Learning & Applications — student track	Strong fit	Moderate
ACM SAC	Conference	Applied Computing — Data Mining track	Strong fit	Moderate
ICICT	Conference	Innovative Computing & Communications	Good fit	Accessible
Expert Systems with Applications	Journal	AI/ML applied systems (Elsevier)	Strong fit	Stretch goal
University Research Symposium	Internal	Departmental showcase	Perfect fit	Guaranteed

Research Paper Writing Roadmap

Section	Content	Written In	Draft By	Final By
1. Introduction	Motivation, research gap, contributions	Week 1	End W1	End W6
2. Related Work	CRM scoring, rule vs ML, microservices, data sparsity	Week 2	End W2 (partial)	End W7
3. System Architecture	NestJS, FastAPI, n8n, data flow	Week 3–4	End W4	End W7
4. Methodology	Dataset, models, evaluation framework, scoring rules	Week 3–5	End W5	End W7
5. Results	H1, H2, H3 findings, tables 2–5, figures 2–3	Week 5–6	End W6	End W7
6. Discussion	Interpretation, implications, limitations	Week 6	End W6	End W8
7. Conclusion	Summary, future work	Week 8	End W8	End W8
Abstract	150–200 words (written LAST after experiments)	Week 8	End W8	End W8

The paper is written WHILE you build — not after. Every week has a research writing task. By the time Week 7 arrives you should have a rough draft of every section. Week 7 and 8 are for polishing, not writing from scratch.

Master Project Checklist — All 8 Weeks

Week	Key Deliverable	Done?	Owner
W1	Docker-compose runs all services on both laptops	☐	Both
W1	Prisma schema migrated (6 tables) — research-critical fields confirmed	☐	A
W1	synthetic_leads.json: 1000 leads, sigma=5/10/15 variants ALL generated	☐	A
W1	React app + Tailwind + Redux confirmed working at localhost:5173	☐	B
W1	Wireframes PDF (4 screens) shared + approved by Person A	☐	B
W1	research_notes.md + research_question.md (200-word gap) committed	☐	Both
W2	All 7 API endpoints return correct responses	☐	A
W2	Swagger UI live at localhost:3000/api/docs	☐	A
W2	LoggingInterceptor logs latencyMs on every request (H3 data collection active)	☐	A
W2	Lead List + Filter Sidebar + Redux store + Axios interceptors working	☐	B
W2	Problem Statement 300-word draft + ADR documents complete	☐	Both
W3	scoring_rules table seeded with all 10 rules	☐	A
W3	lead.created event triggers scoring + saves latencyMs to DB	☐	A
W3	Unit tests: 80%+ coverage on scoring module	☐	A
W3	Score badges (HOT/WARM/COLD) visible on Lead List	☐	B
W3	Methodology Section 4.2 draft + scoring rules table with justifications	☐	Both
W4	/score/compare returns 3 scores + agreement + all latencies	☐	A
W4	scoring_comparisons table populates on every lead creation	☐	A
W4	3 n8n workflow JSONs in /n8n-workflows/ folder	☐	A
W4	Model Comparison toggle (Rule/LR/RF) working in Lead Detail	☐	B
W4	Methodology 4.3 + 4.4 drafts + Table 1 complete	☐	Both
W5	sparsity_results.json: 7 sizes × 5 runs × 3 models (mean ± std)	☐	A
W5	Sigma sensitivity: sparsity results for sigma=5, 10, 15 all saved	☐	A
W5	metrics_sample.json: P, R, F1, AUC-ROC all 3 models committed	☐	Both

W5	Figure 2: sparsity curve with error bars saved as PNG	☐	Both
W5	Analytics Dashboard: 6+ charts live	☐	B
W5	Table 2 (accuracy) + Table 3 (latency — measured values only, no ranges)	☐	Both
W6	results_canonical.json + sparsity_canonical.json locked + tagged v0.6	☐	Both
W6	Full end-to-end flow: lead → score → n8n alert working	☐	A
W6	Discussion Section: H1, H2 (Table 5 interpretability checklist), H3	☐	Both
W6	Table 5 (interpretability checklist 3-dimension scoring) complete	☐	Both
W6	Figures 1, 2 (with error bars), 3 (confusion matrices) all drafted as PNG	☐	Both
W7	Backend: 80%+ integration test coverage	☐	A
W7	Load test report + security headers + Docker container optimised	☐	A
W7	pytest passes for all 6 FastAPI endpoints	☐	A
W7	Cypress: 15+ E2E scenarios passing + accessibility audit + mobile fixes	☐	B
W7	Performance profiling complete + bundle size optimised	☐	B
W7	Related Work Section 2: 15+ citations in 5 clusters	☐	Both
W7	Full paper draft sent to supervisor for review	☐	Both
W8	README enables zero-friction setup + API guide + n8n workflow docs	☐	A
W8	Backend demo video (5 min) + Frontend demo video (5 min) recorded	☐	Both
W8	Project showcase video (10 min, narrated) recorded	☐	Both
W8	User manual (annotated screenshots) + Handover notes committed	☐	B
W8	Abstract (150–200 words, written last with actual findings) complete	☐	Both
W8	Paper formatted to IEEE template in Overleaf — 6–8 pages	☐	Both
W8	Final project report (15–20 pages) complete	☐	Both
W8	15-slide presentation deck complete	☐	B
W8	Target conference identified + submission package ready	☐	Both